

OpenStack, the Day After Tomorrow

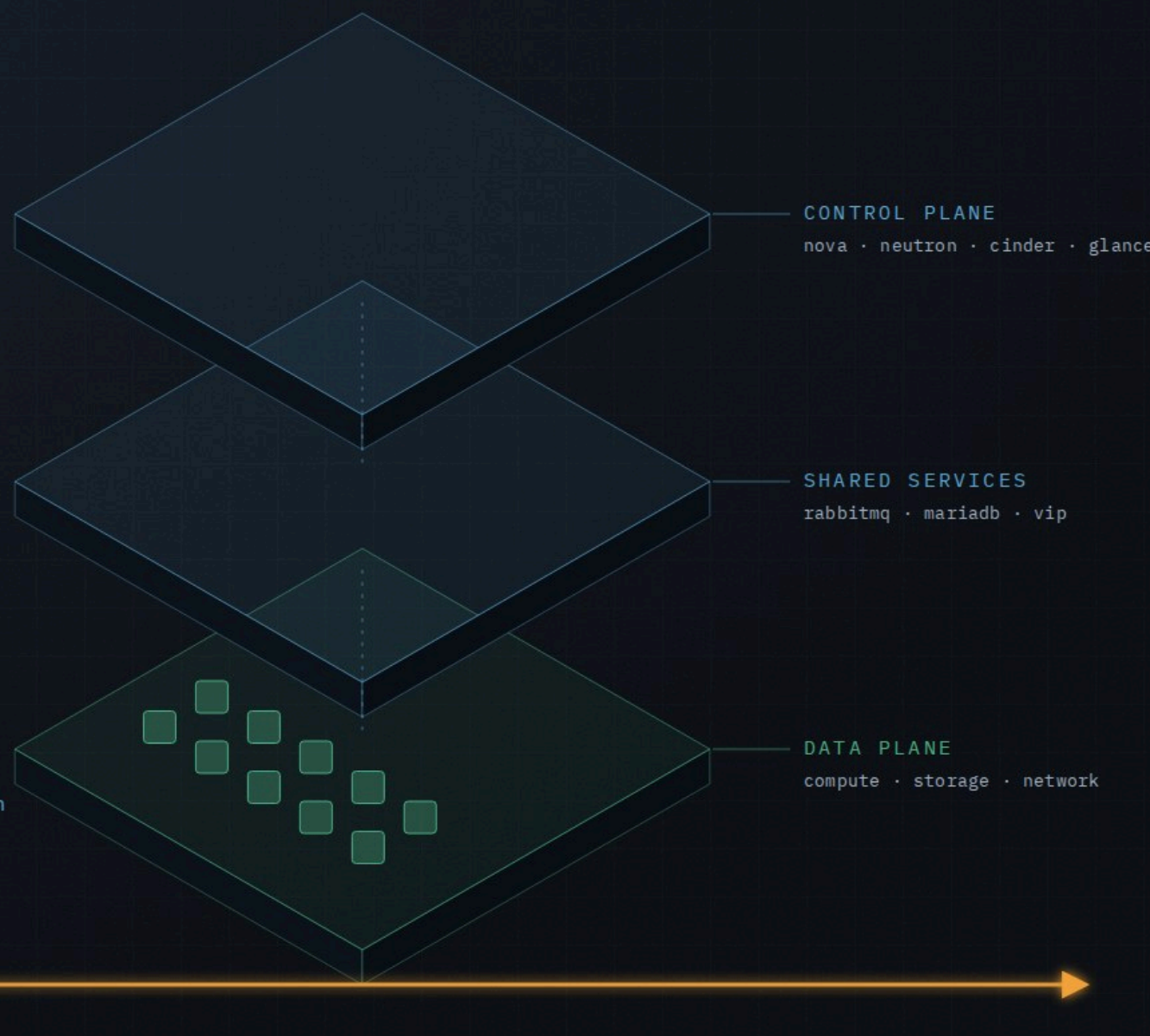
Short Guide

The ideas, principles and reasoning behind operating mission-critical OpenStack platforms — in eighteen illustrated pages.

Soufian Zaouam

companion to the full book

Observe → Understand → Prioritize → Decide → Act → Verify → Learn



An illustrated introduction to operating mission-critical OpenStack platforms: the ideas, the principles and the reasoning behind the full book.

This guide is a short, illustrated companion to [OpenStack, the Day After Tomorrow](#). It does not replace the book and it is not a command reference. It sets out, in eighteen pages, how the book thinks about operating OpenStack once it carries workloads that matter: what goes wrong, what to watch, how to decide, and what the organization around the platform has to become.

How to read it

- Each page carries one idea and one figure. The figure is meant to be read, not admired.
- Short quotations are taken verbatim from the book and marked with their chapter.
- Technical details appear only when they serve an idea. The book gives the depth, the sources and the field cases.
- Engineers can read it in order. Managers can start with pages 02, 03, 09, 15, 16 and 17.

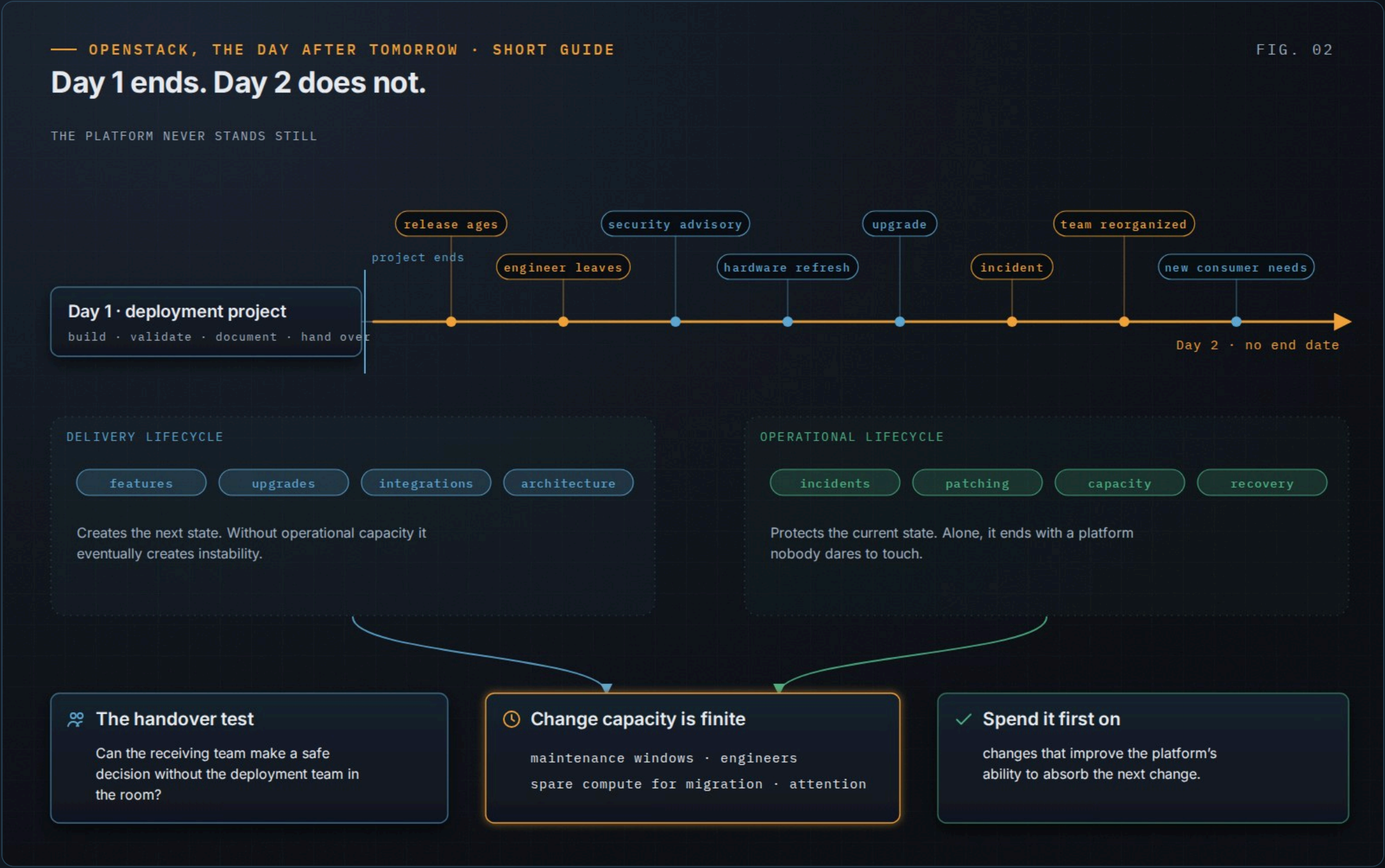
Contents

- 02 The real challenge starts after deployment
- 03 The enemy is loss of control
- 04 The operational mindset: understand before acting
- 05 Read the signals before touching the platform
- 06 Investigate before changing
- 07 Understand the dependencies
- 08 Stability before evolution
- 09 Production is risk management
- 10 Controlled change
- 11 Recovery is a feature
- 12 Technical debt: understood, controlled, decided
- 13 Security, upgrades and maintenance
- 14 Operational memory and ownership
- 15 A team, not a collection of experts
- 16 Decisions from the field
- 17 The operating principles
- 18 From deployment to operation

DAY 1 ENDS. DAY 2 DOES NOT.

02 · The real challenge starts after deployment

A deployment project has an end date. Production does not. The difficult part of OpenStack is not making it work once, but keeping it under control after the people who built it have moved on.



Two lifecycles, one platform

The platform lives two intertwined lives. The **delivery lifecycle** brings features, upgrades and integrations; the **operational lifecycle** absorbs incidents, maintenance, patching, capacity and recovery. A team that only delivers eventually creates instability; a team that only protects the current state eventually owns a platform nobody dares to touch. Day 2 needs an explicit balance between the two, decided rather than left to whoever shouts loudest.

The handover is a capability transfer

A runbook does not transfer the reasons behind a RabbitMQ topology, a constrained Nova setting or a critical network path. The real test of a handover is whether the receiving organization can make a safe decision without the deployment team in the room. When it cannot, the organization has not created an operating model; it has extended the deployment team into production.

Change capacity is finite

Maintenance windows, engineers, spare compute for live migration and attention are all limited. The book calls this **change capacity** and argues for spending it first on changes that improve the platform's ability to absorb the next change.

PRINCIPLE

"The measure of a successful deployment is whether the platform remains operable when the deployment experts are no longer present."

Chapter 1

CONTROL, NOT PERFECTION

03 · The enemy is loss of control

Availability is what the business sees today. Control is what decides whether the organization can still understand, decide, intervene, recover and evolve tomorrow.



Four capabilities

The book organizes Day-2 operations around four capabilities. **Visibility**: knowing what is happening. **Stability**: protecting what matters and keeping a trusted operating state. **Organization**: making knowledge and capability survive people. **Governance**: making decisions with the right evidence, authority and accountability. Each one fails differently without the others: visibility without understanding produces information without action; stability without organization depends on heroes; governance without evidence becomes bureaucracy.

Control erodes silently

A key engineer leaves. A new customization is added. A release reaches end of life. The documentation goes stale. None of these raises an alert, and all of them make the next failure harder to understand or recover from. That is why control management looks for the conditions of future failure, not only for failures that already happened.

Where to invest

Control is judged against consequence. The objective is not to bring every component to the same level, but to find where low control meets high business impact.

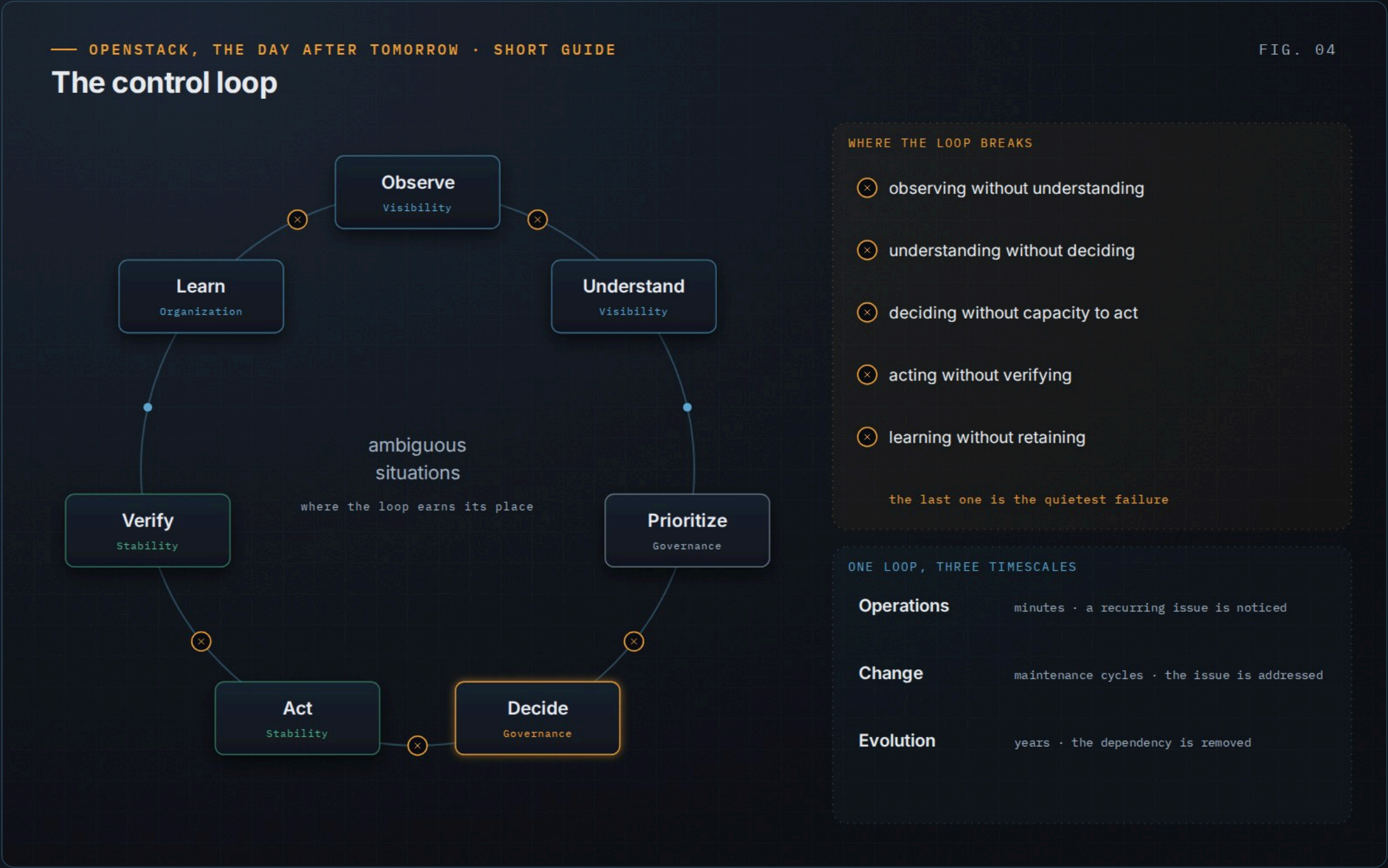
FROM THE BOOK

"Availability is an outcome; control is a capability"

Chapter 2

04 · The operational mindset: understand before acting

The book connects the four capabilities through one loop. It is deliberately simple; its value comes from applying it consistently when the platform is under pressure.



Observe → Understand → Prioritize → Decide → Act → Verify → Learn

Visibility provides evidence, but evidence is not understanding. Understanding produces hypotheses and constraints. Prioritization brings in business impact. Decision rights determine who chooses. Action stays within understood boundaries. Verification checks the outcome, because technical success is not service success. Learning turns the result into organizational capability.

Where the loop breaks

- observing without understanding;
- understanding without deciding;
- deciding without the capacity to act;
- acting without verifying;
- learning without retaining — the quietest failure.

Proportional, not bureaucratic

A routine request goes from observation to action in minutes. The loop earns its place when the answer is not obvious, and it must work at human speed: lightweight for routine work, increasingly explicit as consequence and uncertainty grow.

FROM THE BOOK

“If nothing changes after an incident, the organization should be able to explain why the existing controls were already sufficient.”

Chapter 4

05 · Read the signals before touching the platform

More data does not create visibility. Visibility starts with operational questions: what is affected, which dependencies are shared, what changed, and who owns it.



HYPOTHESES, EVIDENCE, ROOT CAUSE

06 • Investigate before changing

A failed VM creation says what the user experienced. It does not say which of the services and dependencies in the chain caused it.



Facts, hypotheses, assumptions

Operational reasoning separates what is observed, what explains it and what is merely believed. OpenStack produces plausible explanations in abundance: a scheduling problem looks like a Nova problem, a timeout looks like a Neutron problem. Good investigation tries to disprove hypotheses as well as confirm them, and reduces the experimental surface — one host, one workload pattern, one dependency observed without changing anything else.

Evidence proportional to consequence

A read-only diagnostic needs little proof; a production database intervention needs much more. The burden of proof rises with blast radius and irreversibility. When uncertainty is high and waiting is cheap, collecting evidence is safer than changing the platform.

A request is not a diagnosis

“Restart it”, “move these VMs”, “automate this” should each be translated back into the problem they are meant to solve before anyone acts. An incident resolved quickly but never understood tends to come back and consume the same capacity again.

FROM THE BOOK

“Uncertainty is a risk signal in its own right.”

Chapter 9

OPENSTACK IS A DISTRIBUTED SYSTEM

07 · Understand the dependencies

OpenStack is not one program but a set of cooperating services, each with its own database, configuration and failure modes, resting on a message bus, a database cluster, storage and networks that are not OpenStack at all.



What one VM creation traverses

Keystone authenticates. The Nova API records the request; the conductor coordinates; the scheduler asks Placement for capacity; the compute pulls the image from Glance and asks Neutron for a port, which the agent or OVN controller wires on the host. Nova waits for the `network-vif-plugged` event before finishing the boot. A failure anywhere in that chain looks, to the user, like “Nova is broken”.

Control plane and data plane fail separately

If the APIs, the message bus or the databases stop, existing instances keep running, keep their storage and keep their traffic. What is lost is the ability to create, change or move resources. That separation is what makes “stop the APIs, protect the workloads” a real option in an incident rather than a euphemism for an outage.

Assess capabilities, not components

An inventory lists what exists. The useful question is whether the organization can operate the resulting service: follow a real user journey through every dependency and ask, at each step, where the signal is, who owns it and how it is recovered. Gaps appear at team boundaries.

FROM THE BOOK

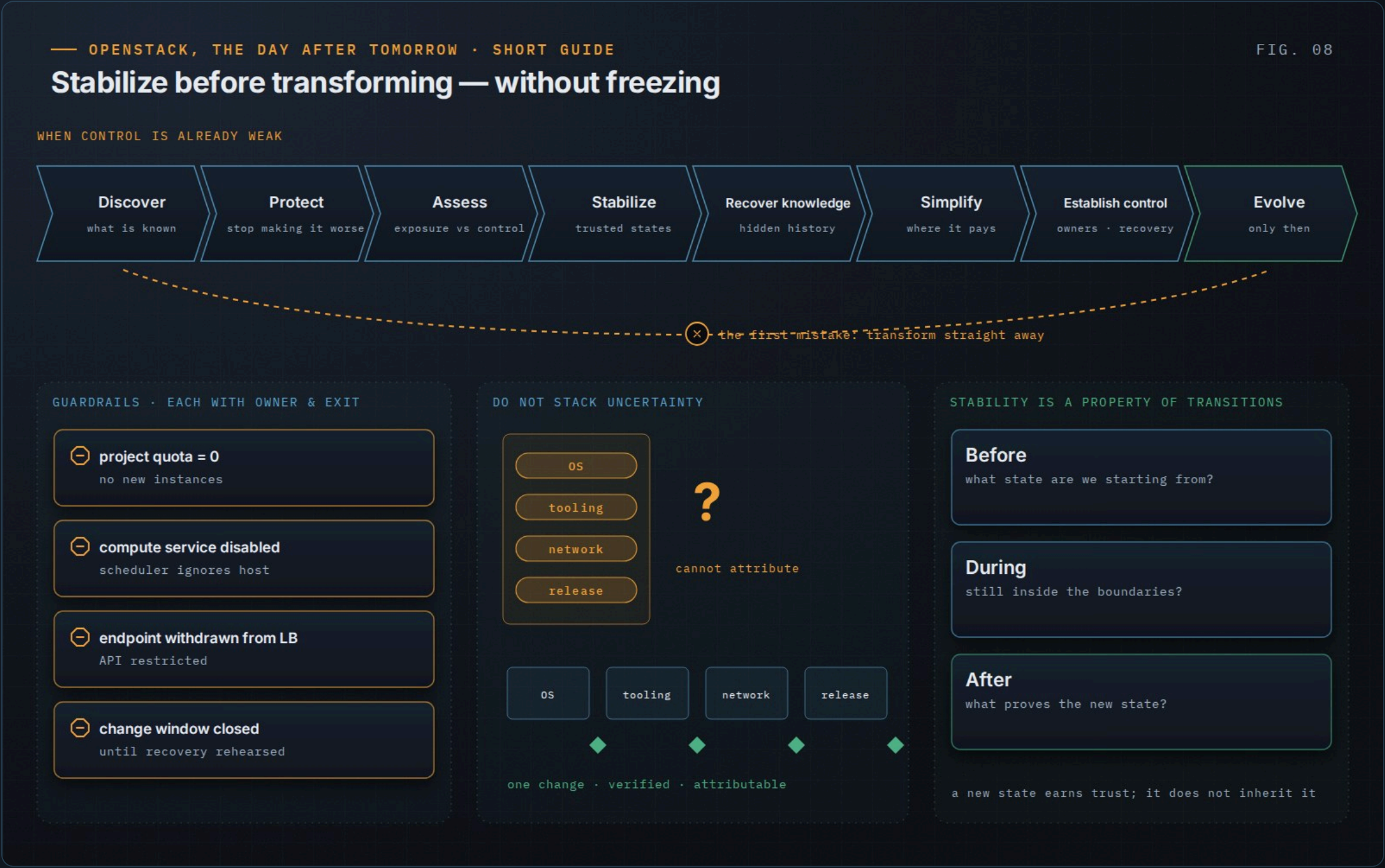
“an instance that sits in BUILD until vif_plugging_timeout expires is usually a networking problem wearing a compute symptom.”

Chapter 5

DO NOT STACK UNCERTAINTY

08 · Stability before evolution

When a platform is already unstable, adding major change makes diagnosis harder and widens the blast radius of every problem. Stabilize first — but not forever.



Restore before you transform

When control is already weak, the first mistake is usually to start transforming. The book proposes a sequence that deliberately slows the urge to fix visible symptoms: Discover, Protect, Assess, Stabilize, Recover knowledge, Simplify, Establish control — and only then Evolve. Guardrails used on the way (a quota set to zero, a disabled compute service, an endpoint withdrawn from the load balancer) each need an owner and a removal condition.

The caveat

Some platforms are so far behind — unmaintained releases, unpatched vulnerabilities — that the transformation is the only available stabilization. The rule is not “never move while things are difficult”. It is “do not stack uncertainty”: changing the OS, the deployment tooling, the network and the OpenStack release in one window makes any failure impossible to attribute.

Stability lives in transitions

Before a change: what state are we starting from? During: how do we know we are still inside boundaries? After: what evidence tells us the new state can be trusted? A new state has to earn trust; it does not inherit it.

FROM THE BOOK

"Stabilization that has no exit condition becomes the new architecture."

Chapter 7

EVERY TECHNICAL DECISION HAS A BUSINESS CONSEQUENCE

09 · Production is risk management

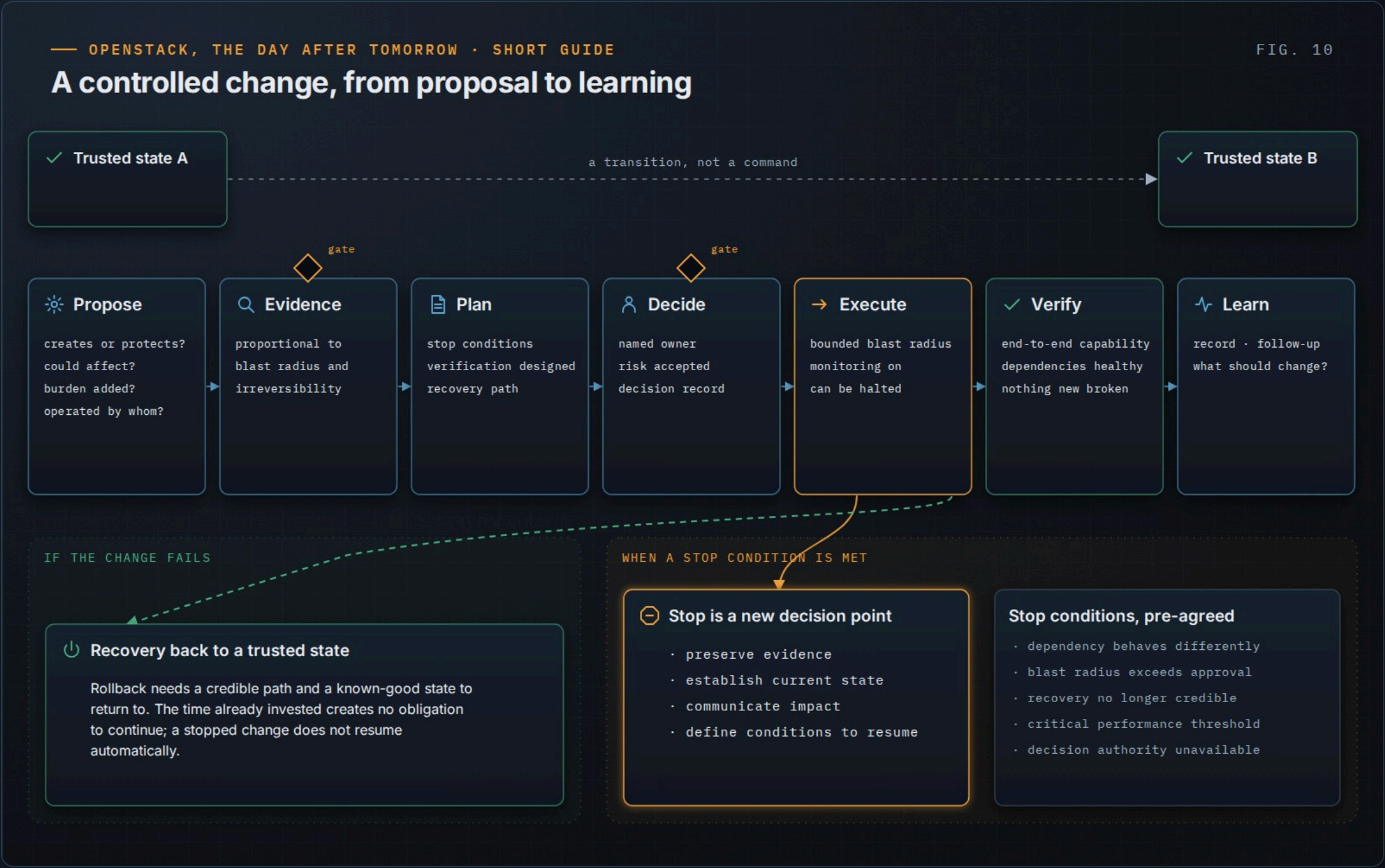
A change that looks local to Nova may reach Placement, Neutron, the message bus, the databases, the hypervisors and the consumers' own workflows. Service boundaries are not business boundaries.



UNDERSTOOD · RECORDED · VERIFIABLE · RECOVERABLE

10 · Controlled change

Approval alone does not make a decision controlled. A controlled change knows what it risks, how failure will be detected and how the platform returns to a trusted state.



Before

Every proposal answers four questions: what capability it creates or protects, what existing capability it could affect, what operational burden it adds, and what it will take to operate once the delivery team has moved on. Verification and stop conditions are designed before execution, not improvised in the middle of it.

During

Blast radius is bounded, monitoring is in place and the conditions to stop are concrete: a dependency behaves differently, the impact exceeds the approved boundary, recovery is no longer credible, the decision authority is unavailable. The time already invested creates no obligation to continue, and a stopped change does not resume automatically — resumption is a new decision.

After

A change is complete when the capability it served is verified end to end, not when the command returns. A proportional decision record keeps the objective, evidence, uncertainty, options considered, rejected options, recovery path and follow-up, so that the next team does not mistake a deliberate choice for inertia.

PRINCIPLE

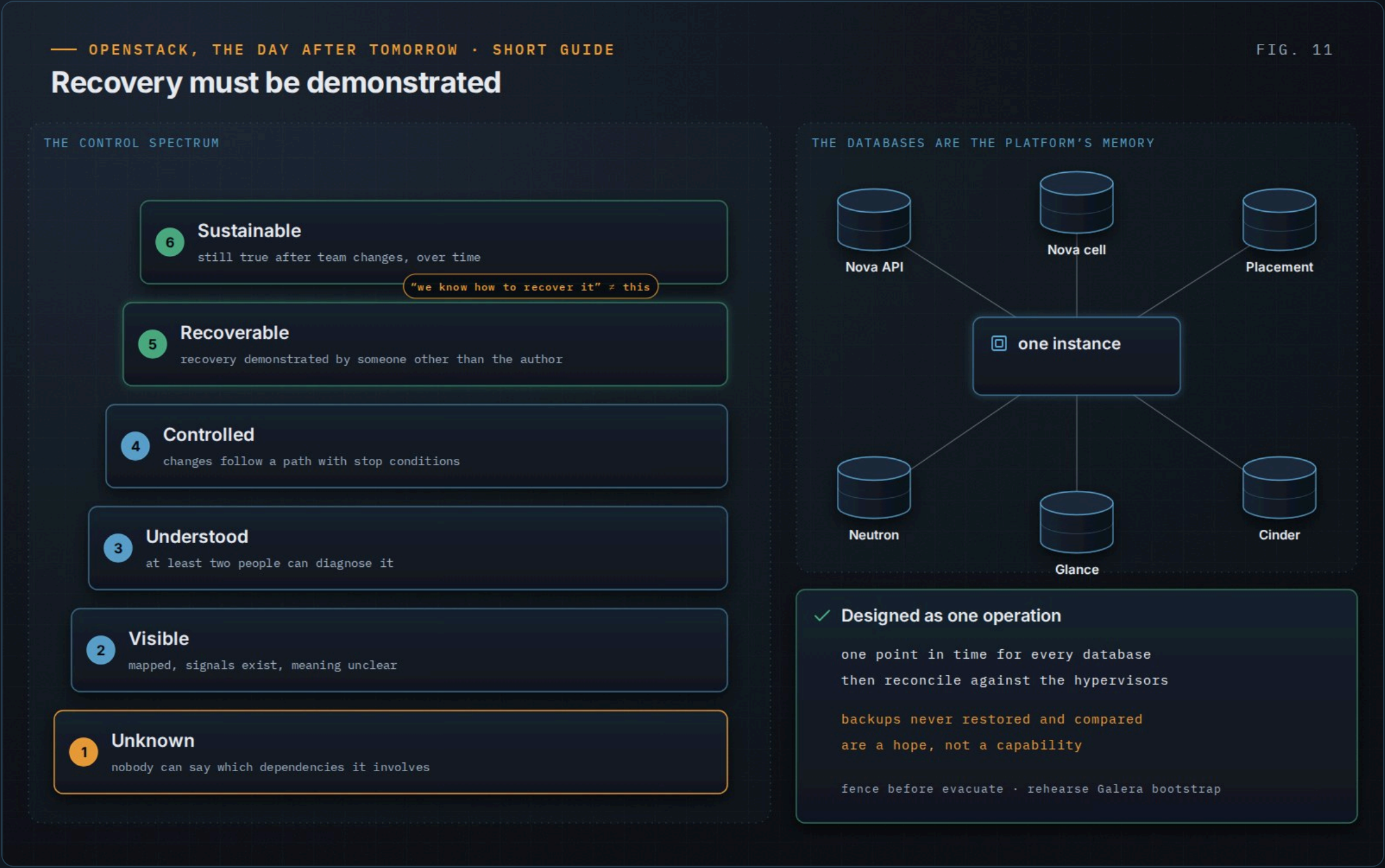
“Before changing a mission-critical OpenStack platform, understand enough to know what you are risking, how you will detect failure, and how you will recover if you are wrong.”

Chapter 9

AVAILABILITY IS NOT RECOVERABILITY

11 · Recovery is a feature

A service can run while the organization is unable to recover it under realistic failure conditions. Recovery starts from a trusted state, is designed around capabilities and must be demonstrated.



Demonstrated, not documented

"We know how to recover RabbitMQ" is not enough if the procedure lives in one memory. The book's control spectrum makes the difference explicit: a capability is only **Recoverable** when recovery has been demonstrated by someone other than the author of the procedure, within a time the business accepts — and **Sustainable** when that still holds after team changes.

The databases are the platform's memory

The state of one instance is spread across Nova, Placement, Neutron, Cinder and Glance databases. Restoring them to different points in time produces a platform that disagrees with itself and with the hypervisors. Recovery is one operation, one point in time, followed by reconciliation against reality.

Recovery is a change under pressure

The same disciplines apply, harder. A missing heartbeat does not prove a compute host is dead: fence it through out-of-band management before evacuating, or two copies of one server end up writing to the same volume. And sometimes the safest recovery of a disposable environment whose state nobody understands is a rebuild against a written baseline.

PRINCIPLE

"Recovery is a feature because a platform is not operational if the organization cannot demonstrate how it returns to a trusted state."

Chapter 10

DEBT IS NOT FAILURE. FORGOTTEN DEBT IS.

12 · Technical debt: understood, controlled, decided

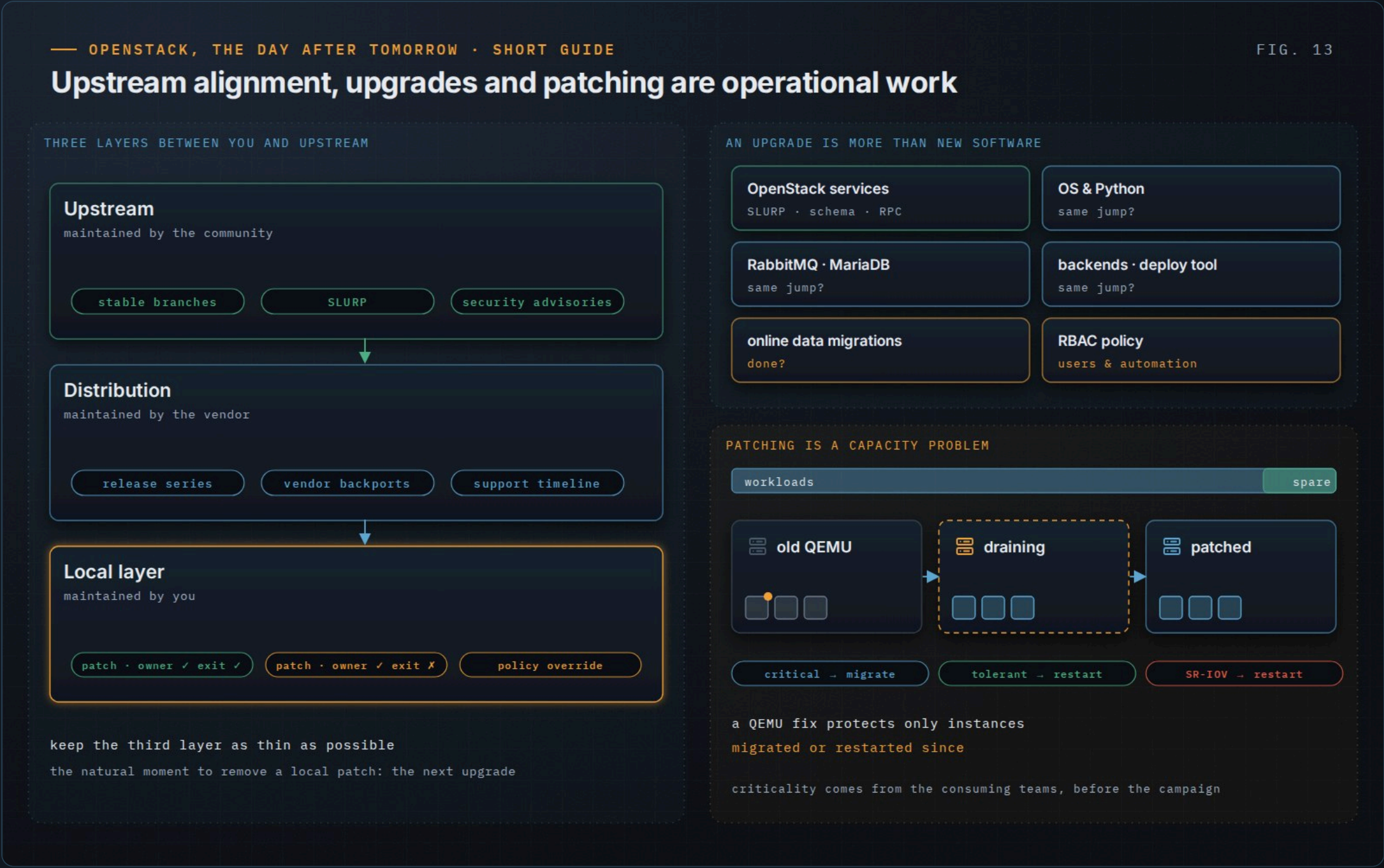
Platforms rarely become hard to operate because someone designed them that way. Complexity accumulates through decisions that were each rational at the time.



STAYING CLOSE TO UPSTREAM IS OPERATIONAL CONTROL

13 · Security, upgrades and maintenance

A platform cannot stay unchanged indefinitely. Releases age, dependencies stop being maintained and the cost of standing still becomes part of the risk.



Know your three layers

Upstream, distribution, local. The vendor’s backports are maintained by the vendor; the layer that creates control debt is the local one on top. A backport can be the right decision — typically for a security advisory whose fix does not exist for the running series — but only with an owner, an origin, a test and an exit condition, usually the next upgrade.

Upgrades are organizational readiness

SLURP lets the OpenStack services skip a release; the operating system, deployment tool, RabbitMQ, MariaDB and backends must support the same jump, and online data migrations must be complete. Readiness also means recovery coverage, capacity, ownership and policy: new default RBAC can change what users and automation are allowed to do.

Security is platform work

Patching a hypervisor is a placement and capacity problem: a QEMU fix only protects instances migrated or restarted since, and spare capacity decides what can be live-migrated. Certificates and Fernet keys age silently and belong on the first screen and in rehearsed rotations.

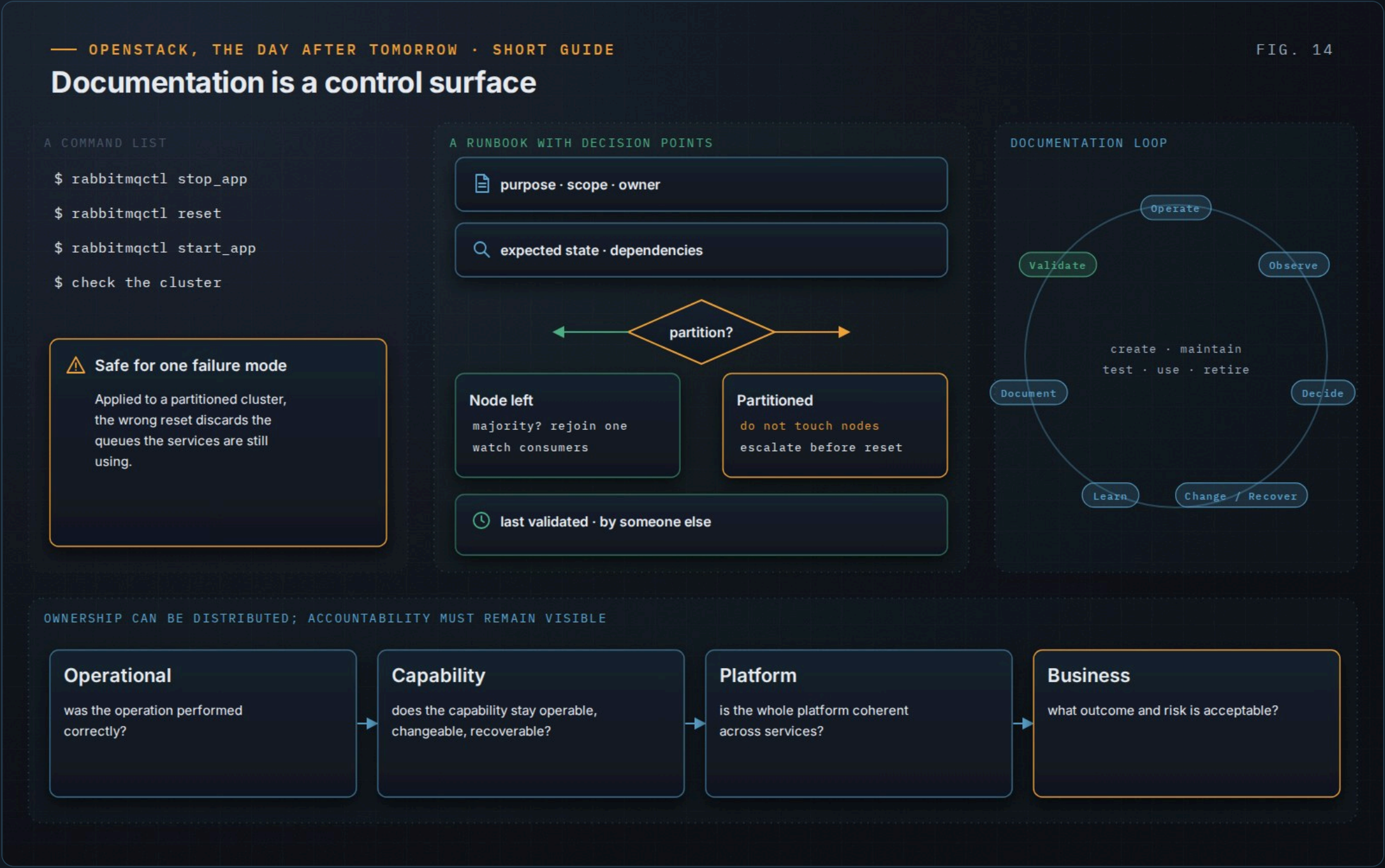
PRINCIPLE

“Security requirements define what must be achieved. Platform capacity and business criticality determine how safely it can be achieved.”

Chapter 20

14 · Operational memory and ownership

Operating a critical platform does not rest on technology alone. Knowledge, ownership and accountability are part of the system — and they fail too.



Documentation preserves reasoning

A procedure that says “restart component X” is worth less than one that also says why, what state to expect, which dependencies matter, when to stop and what must never be done without escalation. The runbook for a RabbitMQ node that has left the cluster is not the runbook for a partitioned cluster, even though both start from the same alert. Decision records keep the why, including rejected options.

Tested by someone else

A new engineer executing a real procedure alone, with a senior engineer watching but not helping, exposes every gap the team fills without noticing. Documentation that cannot be used by another engineer is not yet operational knowledge.

Ownership and accountability

Ownership means maintaining a capability; accountability means answering for its outcome and risk. Ownership can be distributed; accountability must remain visible — at the operational, capability and platform levels, distinct from business accountability. Organizational boundaries must not become operational boundaries.

FROM THE BOOK

“Every one of those gaps was knowledge the team had; none of it was on the page.”

Chapter 11 – From the field

15 · A team, not a collection of experts

Strong experts create their own operational risk precisely because they are good. The purpose of expertise is to make the organization less dependent on it for routine work.



Redundancy of knowledge

Not two people on every task, but no critical capability with a single point of failure. One person may stay the deepest expert while several others can operate and recover. Naming a secondary is not enough: the secondary must perform real operations, join incidents, review changes and be able to explain the recovery.

Identity, workload, leadership

A team with a shared purpose challenges unsafe changes and protects the platform beyond its tickets; a team that sees itself as a ticket queue has no reason to improve the system that generates them. Overload is a reliability risk: insufficient capacity concentrates expertise, which increases dependency and pressure, which leaves even less room for improvement. Leadership’s role is to keep the long-term view when short-term delivery pressure pushes against it.

PRINCIPLE

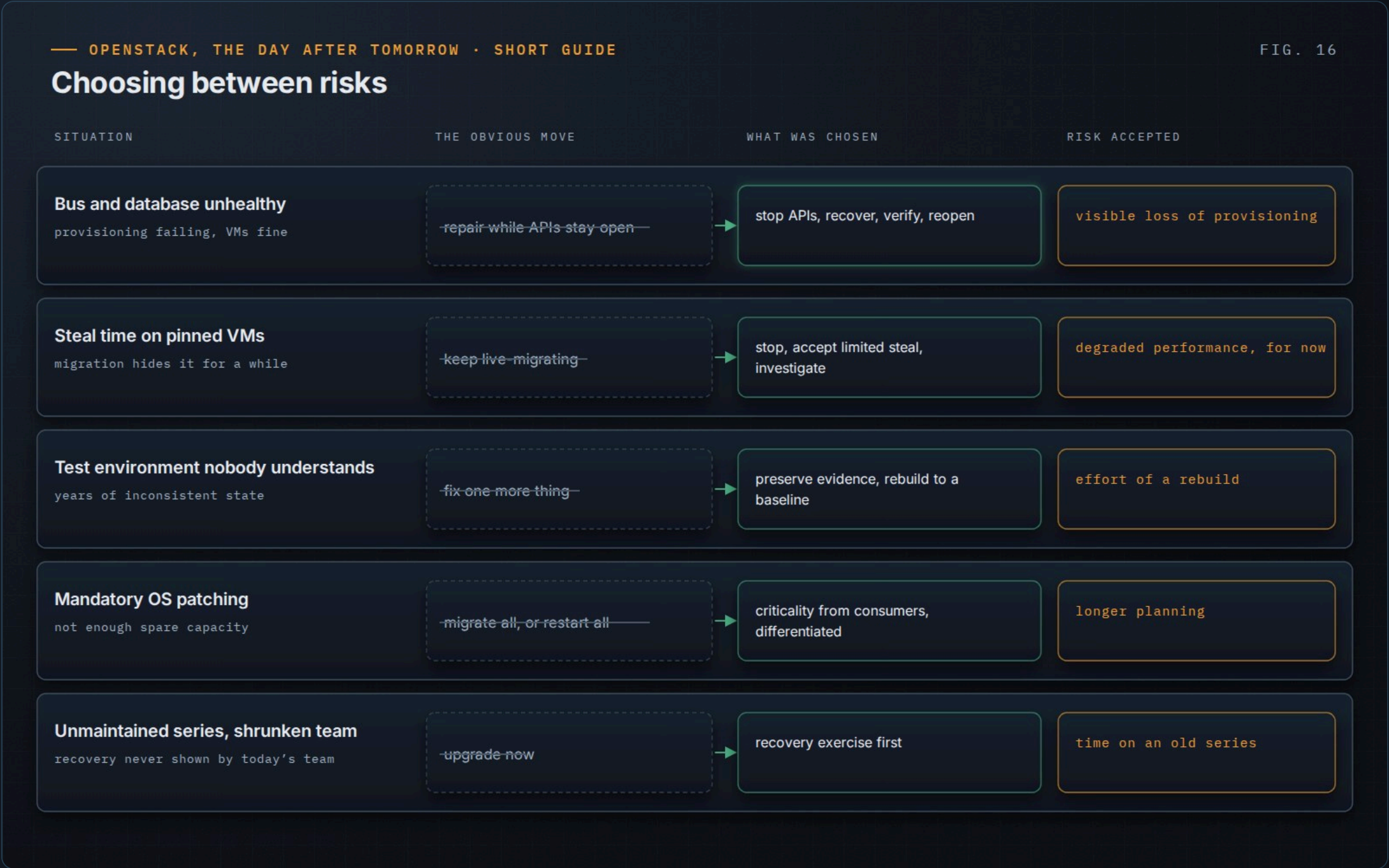
“A team is resilient when the platform remains operable after its experts change.”

Chapter 12

OPERATING MEANS CHOOSING BETWEEN RISKS

16 · Decisions from the field

These are not recipes. They are situations from the book in which technical evidence, business impact and operational constraints collided, and in which the safe choice was not the obvious one.



Stop, to keep control

RabbitMQ and the Galera cluster were unhealthy; VM creation was failing, running workloads were not. The team stopped the APIs, restarted the bus after purging its queues, then MariaDB, verified end to end and only then reopened. A visible, bounded loss of provisioning instead of an unstable control plane accepting requests.

Investigate instead of repeating

Pinned instances showed steal time; live migration made it disappear, until it came back. The team stopped migrating the same workloads, accepted limited steal, and investigated the cause instead of relocating the symptom.

Rebuild what nobody understands

A shared test environment had accumulated years of inconsistent state. Evidence was preserved, the intended baseline written down, and the environment rebuilt and validated against it. Production changes that equation entirely.

Patch within real capacity

There was not enough spare capacity to live-migrate everything before OS patching. Consuming teams supplied criticality and acceptable downtime; critical workloads were migrated, others restarted in coordination.

Rehearse before upgrading

In the book's decision model, an unmaintained platform's upgrade ranking hinged on one fact: nobody still on the team had demonstrated recovery. The first action was a recovery exercise, not an upgrade.

PRINCIPLE

"Stopping is not giving up control. In the right circumstances, it is how control is preserved."

Chapter 19

17 • The operating principles

The book states its operating rules as principles. They are recommendations, not laws of nature; a different platform and organization may weigh them differently. Together they describe one way of thinking about production.

VISIBILITY

The purpose of observability is to make the platform understandable enough to support the right decision at the right time.

CHAPTER 8

STABILITY

Before changing a mission-critical OpenStack platform, understand enough to know what you are risking, how you will detect failure, and how you will recover if you are wrong.

CHAPTER 9

Stopping is not giving up control. In the right circumstances, it is how control is preserved.

CHAPTER 19

Recovery is a feature because a platform is not operational if the organization cannot demonstrate how it returns to a trusted state.

CHAPTER 10

ORGANIZATION

The measure of a successful deployment is whether the platform remains operable when the deployment experts are no longer present.

CHAPTER 1

A team is resilient when the platform remains operable after its experts change.

CHAPTER 12

Documentation is operational memory only when the organization has a deliberate way to create it, maintain it, test it, use it and retire it.

CHAPTER 11

GOVERNANCE

Everyone can challenge. Someone must decide.

CHAPTER 13

Complexity is part of the operational risk of an OpenStack platform, and it must earn its place.

CHAPTER 3

A workaround is safe only while the risk it introduces remains smaller than the risk it removes.

CHAPTER 15

EVOLUTION

Stay as close to upstream as practical.

INTRODUCTION · CHAPTER 14

The objective is the right level of automation for the platform the organization can actually operate, which is rarely the maximum.

CHAPTER 16

Security requirements define what must be achieved. Platform capacity and business criticality determine how safely it can be achieved.

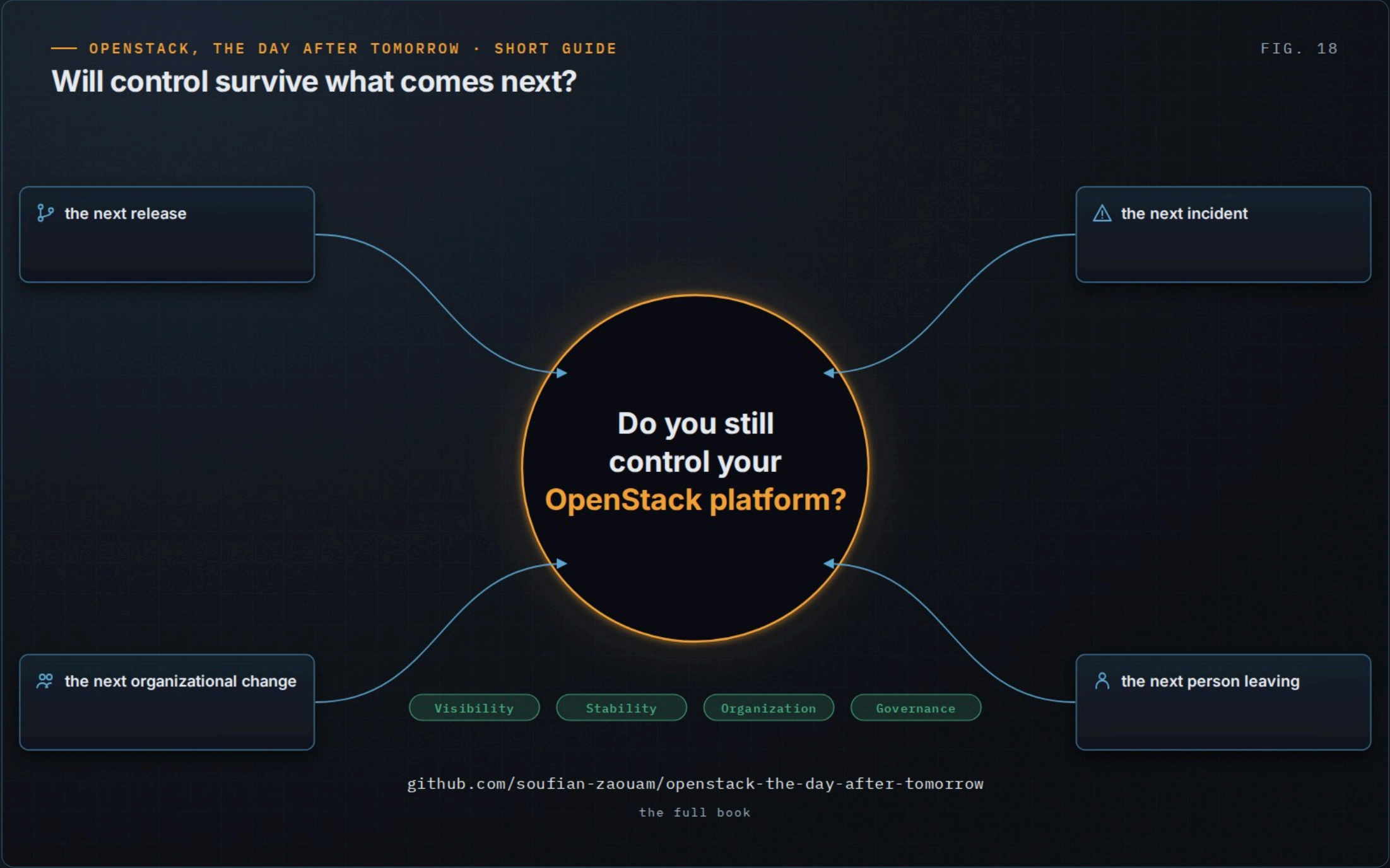
CHAPTER 20

A successful OpenStack evolution is a platform whose future can still be operated deliberately, not merely a newer one.

CHAPTER 18

18 · From deployment to operation

Deployment creates an OpenStack platform. Day 2 creates the organization’s relationship with it — and that relationship decides whether the platform can still be patched, recovered, upgraded, simplified and challenged.



The questions that carry the discipline

- What do we know? What remains uncertain?
- What does the business depend on? What are we protecting?
- What could this action break? Can we recover?
- Who decides? What are we accepting by choosing this option?
- What will we learn from the outcome?

The ability to say no

To a change whose risk is not understood, to unnecessary complexity, to automation without sufficient value, to a workaround that has become dangerous, to a transformation the organization cannot yet control — and sometimes to the pressure to restore everything immediately when protecting the existing platform is the safer choice.

Go further

This guide gives the ideas. The full book develops them: the assessment worksheet and control spectrum, the measures the platform can produce itself, the technical notes on Nova, Neutron, RabbitMQ, Galera, SLURP and live migration, the decision framework with its gates and weighted model, and the two field cases worked through in full.

Read the full book: <https://github.com/soufian-zaouam/openstack-the-day-after-tomorrow>

FROM THE BOOK

“OpenStack does not end at deployment. It begins there. The platform will change, people will change, the organization will change. The measure of operational maturity is whether control survives those changes.”

Conclusion